PSAI_129

Implementation of the new hash function to BLS12 curves

- **Team Name:** Dmitrii Koshelev
- **Payment Address:** 0x050240CB2f5F7f7aA8AA10a5A9AE36c784AEdE49 (ERC20 USDC)
- **Level:** 2

Project Overview :page_facing_up:

Overview

Please provide the following:

- If the name of your project is not descriptive, a tag line (one sentence summary).

Implementation of my hash function in Sage in the general case and in Rust specifically for the curve BLS12-381.

- A brief description of your project.

Recently, my article https://link.springer.com/article/10.1007/s10623-022-01012-8 was published in the quite prestigious cryptographic journal “Designs, Codes and Cryptography”. This article provides a new hash function (indifferentiable from a random oracle) to the subgroup G1 of many elliptic curves of j-invariant 0, including most pairing-friendly curves of the family BLS12 (Barreto-Lynn-Scott) of embedding degree 12. Such curves and hash functions are actively used in blockchains, namely in the BLS (Boneh-Lynn-Shacham) aggregate signature. According to the web page https://wiki.polkadot.network/docs/learn-keys, the BLS signature will be in particular used in Polkadot.

My hash function is much faster than previous state-of-the-art ones, including the Wahby-Boneh indirect map to the curve BLS12-381. This curve is a de facto standard in today’s real-world pairing-based cryptography. The indifferentiable Wahby-Boneh hash function requires to extract two square roots (i.e., to compute two exponentiations) in the basic field Fp. In comparison, the new hash function extracts only one cubic root, which can be also expressed via one exponentiation in Fp. And according to Section 1.1 of my other article https://eprint.iacr.org/2021/1082 the exponentiations for square and cubic roots in Fp have short addition chains of about the same length.

I have already checked the correctness of my results in the computer algebra system Magma. The new project is dedicated to an implementation of the new hash function to BLS12-381 in Rust. This project is a follow-up of the previous one concerning a proof-of-concept implementation in Sage (see #825). I wrote such an implementation, but I didn’t finish the project, because my code does not meet high standards of Web3 Foundation. However, I found a professional developer in Rust who is ready to provide a Rust implementation (based on my Sage one), including tests. Consequently, besides my initial $10,000 reward (level 1), an additional reward for my colleague’s work is required.

- An indication of how your project relates to / integrates into Substrate / Polkadot / Kusama.

According to the web page https://wiki.polkadot.network/docs/learn-keys, the BLS signature will be in particular used in Polkadot.

- An indication of why your team is interested in creating this project.

Because I want to demonstrate to the blockchain society that my research has an immediate practical value.

Project Details

- PoC/MVP or other relevant prior work or research on the topic

My article https://link.springer.com/article/10.1007/s10623-022-01012-8 under the name “Indifferentiable hashing to ordinary elliptic Fq-curves of j=0 with the cost of one exponentiation in Fq”.

Ecosystem Fit

- Where and how does your project fit into the ecosystem?

Hashing to elliptic curves in the BLS (Boneh-Lynn-Shacham) aggregate signature.

- Who is your target audience (parachain/dapp/wallet/UI developers, designers, your own user base, some dapp's userbase, yourself)?

Developers who will use the new hash function in their implementations of cryptographic protocols.

- Are there any other projects similar to yours in the Substrate / Polkadot / Kusama ecosystem?
  - If so, how is your project different?
  - If not, are there similar projects in related ecosystems?

I don't know, because my project is quite extraordinary, in my opinion. Maybe, similar projects (based on recent results of mathematical cryptography) periodically arise in the Ethereum Foundation grants program.

Team :busts_in_silhouette:

Team members

- Name of team leader

Dmitrii Koshelev

- Names of team members

My colleague who is a professional Rust programmer. He prefers to be anonymous.

Contact

- **Contact Name:** Dmitrii Koshelev
- **Contact Email:** dimitri.koshelev@gmail.com
- **Website:** https://www.researchgate.net/profile/Dimitri-Koshelev

Legal Structure

- **Registered Address:** 27B boulevard Jourdan, app. 317, Paris, France
- **Registered Legal Entity:** In this project I am an individual, but I work as a postdoctoral researcher in Telecom Paris (Polytechnic Institute of Paris).

Team's experience

There is the regularly updated CFRG draft on hashing to elliptic curves: https://datatracker.ietf.org/doc/draft-irtf-cfrg-hash-to-curve/. As for me, at the beginning of the last year, as said in https://blog.ethereum.org/2021/11/04/esp-allocation-update-q2-2021/, I won an Ethereum Foundation grant to derive a fast hash function to the subgroup G2 of the pairing-friendly curve BLS12-381. As an outcome of this grant, in Section 1.2 of https://eprint.iacr.org/2021/1082 I prove that for hashing to G2 the Wahby-Boneh map applied only once (not twice) is sufficient to act as a random oracle. Thus we get rid of one exponentiation in the quadratic extension Fp2 or, equivalently, of two exponentiations in the prime field Fp. Besides, I constructed a series of hash functions to other elliptic curves in the articles https://eprint.iacr.org/2021/1034 (accepted in SIAM Journal on Applied Algebra and Geometry), https://link.springer.com/article/10.1007/s12095-021-00478-y (published in Cryptography and Communications), https://eprint.iacr.org/2021/1604 (submitted to Journal of Mathematical Cryptology).

Team Code Repos

- https://github.com/dishport

Team LinkedIn Profiles (if available)

- https://www.linkedin.com/in/dimitri-koshelev/

Development Status :open_book:

- links to your research diary, blog posts, articles, forum discussions or open GitHub issues,

https://ethresear.ch/t/indifferentiable-hashing-to-ordinary-elliptic-fq-curves-of-j-0-with-the-cost-of-one-exponentiation-in-fq/8867

https://github.com/cfrg/draft-irtf-cfrg-hash-to-curve/issues/316

https://github.com/dishport/Indifferentiable-hashing-to-ordinary-elliptic-curves-of-j-0-with-the-cost-of-one-exponentiation

https://link.springer.com/article/10.1007/s10623-022-01012-8

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** 6 months
- **Full-Time Equivalent (FTE):** 1 FTE
- **Total Costs:** 15,000 USD

Milestone 1 — An implementation in Sage

- **Estimated duration:** 3 months
- **FTE:** 1
- **Costs:** 10,000 USD

Number	Deliverable	Specification
0a.	License	GNU GPL
0b.	Documentation	I will provide **inline documentation** of the code.
0c.	Testing Guide	
0d.	Docker	I do not intend to deliver this, because the project is research oriented.
0e.	Article	I will cite the implementation and I will thank Web3 Foundation for its financial support in my new article https://eprint.iacr.org/2021/1082. I submitted this article to Journal of Cryptology. At the moment, it is under review.
1.	Implementation	Sage (non-constant-time) implementation of the hash function described below.

To be definite, let me use the notation of my article https://link.springer.com/article/10.1007/s10623-022-01012-8. The new hash function consists of three components: a classical one : {0,1}^* -> Fp^2, a rational map : Fp^2 -> T(Fp), and an additional map h^: T(Fp) -> E(Fp), where E is a given elliptic Fp-curve of j-invariant 0 and T is a suplementary algebraic threefold. A construction of is represented in Section 5 of the draft https://datatracker.ietf.org/doc/draft-irtf-cfrg-hash-to-curve/. This is the composition of a hash function {0,1}^* -> {0,1}^n for some n N and the subsequent restriction modulo p, hence we can use a standard hash function from one of cryptographic libraries. It remains to implement and h^just as described in my article.

It is also worth noting that the new hash function is relevant for any ordinary (i.e., non-supersingular) elliptic Fq-curve y^2 = x^3 + b (of j-invariant 0) such that the square root of b lies in Fq. Here Fq is an arbitrary finite field (of characteristic p > 3) for which 3 divides q-1, but 27 does not. Therefore, our Sage implementation is in fact relevant for all such curves and fields. The BLS12-381 curve corresponds to the case when q = 10 (mod 27) and b is not a cubic residue in Fq.

Milestone 2 — A non-constant-time implementation in Rust in the case of the BLS12-381 curve

- **Estimated duration:** 3 months
- **FTE:** 1
- **Costs:** 5,000 USD

Number	Deliverable	Specification
0a.	License	Apache License, Version 2.0 and MIT license
0b.	Documentation	I will provide **inline documentation** of the code.
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. I will describe how to run these tests.
0d.	Docker	I do not intend to deliver this, because the project is research oriented.
		I will cite the implementation and I will thank Web3 Foundation for its financial support in my

0e.	Article	new article https://eprint.iacr.org/2021/1082. I submitted this article to Journal of Cryptology. At the moment, it is under review.
1.	Implementation	Rust non-constant-time implementation of the new hash function for BLS12-381. It is suggested to use the library arkworks (https://github.com/arkworks-rs/curves) as a base. In particular, to perform the (unique) exponentiation in Fp an addition chain of quite small length has already been derived in https://github.com/dishport/Some-remarks-on-how-to-hash-faster-onto-elliptic-curves (cf. Section 1.1 of https://eprint.iacr.org/2021/1082).

Future Plans

The new hash function appears to be optimal, because it performs only one exponentiation in the basic field Fq. In other words, since it is highly unlikely that there is a hash function to an elliptic curve without exponentiations at all (even if it is supersingular), the result of my article seems to be unimprovable. Thus I hope that the implementation of my hash function will be interesting not only to you, but also to all blockchain projects that involve the BLS signature.

Additional Information :heavy_plus_sign:

How did you hear about the Grants Program? Web3 Foundation Website

- If there are any other teams who have already contributed (financially) to the project.

No

- Previous grants you may have applied for.

An Ethereum Foundation grant called “fast hash-to-curve research” and represented on the web page https://blog.ethereum.org/2021/11/04/esp-allocation-update-q2-2021/